Practical SBI-17 - Energy-based scoring and flexible refinement of docking runs

Until now we have performed the docking of proteins using three different methods: FTDock, ZDOCK and PathDock. In this practical, we are going to explore two downstream manipulations that can be done on the results of docking programs.

Firstly, we are going to explore the use of an alternative scoring function to re-rank the scoring poses provided by a docking method. From the previous practical you used the native scoring function of each method, but there are other scoring functions available that can be used. To that end, we will use the energy-scoring function of pyDock.

Secondly, as protein are dynamic entities, and so the interaction of proteins, we will apply limited flexibility to refine the docking poses. By applying flexibility, we will refine the dockings (basically by optimizing the side-chain positions), and re-rank the posed base on the refined structures. The FireDock program will be used for the flexible refinement.

VERY IMPORTANT!!!!

KEEP A COPY OF ALL THE FILES GENERATED IN THIS PRACTICAL AS WE ARE GOING TO USE AGAIN (SOME OF THEM ANYWAY) IN THE NEXT PRACTICAL: PRACTICAL SBI-18

1. Energy-based scoring of TolB-Pal complex with PyDock

The TolB and Pal protein complex in E. coli is involved int maintaining the stability of the outer membrane. This complex is important because is the target of the colicins antibiotics (protein antibiotics) using this complex to expedite the entrance in the bacteria. Nuclease colicins competitively recruit TolB using their natively disordered regions (NDRs) to disrupt its complex with Pal, which is thought to trigger translocation of the toxin across a locally destabilized.

We will use pyDock 3.0 to perform a complete docking execution, including rigid-body docking and energy-based scoring. We will generate rigid-body docking poses with ZDOCK for this target, which will be scored with energy-based pyDock scoring function. Then, you will have to make your choice and select what you consider to be the best model from our starting pool of docking poses. Finally, RMSD comparison with the real 3-D complex structure will be done to check the results of our results.

1.1 pyDock 3.0 general syntax

We will use pyDock 3.0 to perform a complete docking execution, including rigid-body docking and energy-based scoring. We will generate rigid-body docking poses with ZDOCK for this target, which will be scored with energy-based pyDock scoring function. Then, you will have to make your choice and select what you consider to be the best model from our starting pool of docking poses. Finally, a RMSD comparison with the real 3-D complex structure will be done to check the results of our dockings. We could in principle run ZDOCK separately as we did in the previous practical and then import the results into pyDock.

All pyDock jobs are launched as follow:

```
pyDock3 dockname module
```

The argument *dockname* (similar to *project name)* is arbitrarily chosen by the user. In our example, we will use **T26** as *dockname*. The different modules that can be used in *pyDock* are listed here:

	module	**Input files**	**Output files**
Docking	*setup*	dockname.ini	dockname_rec.pdb dockname_lig.pdb
	ftdock (or *zdock*)	dockname_rec.pdb dockname_lig.pdb	dockname.ftdock (or dockname.zdock)
	rotftdock (or *rotzdock*)	dockname.ftdock (or dockname.zdock)	dockname.rot
	dockser	dockname_rec.pdb dockname_lig.pdb dockname.rot	dockname.ene
Additional tools	*dockrst*	dockname.ini dockname_rec.pdb dockname_lig.pdb dockname.rot dockname.ene	dockname.eneRST dockname.rst
	patch	dockname_rec.pdb dockname_lig.pdb dockname.rot dockname.ene	dockname.recNIP dockname_rec.pdb.nip dockname.ligNIP dockname_lig.pdb.nip
	oda	X.pdb Y.pdb	dockname_rec.pdb.oda dockname_rec.pdb.oda.ODAtab dockname_lig.pdb.oda dockname_lig.pdb.oda.ODAtab

1.2 Prepare the input files

Firstly, download the coordinates of the unbound proteins from the protein databank (rcsb.org) and extract the relevant chains for the reference complex (2HQS); no need for the unbound complexes:

1C5K, chain A – Unbound TolB
1OAP, chain A – Unbound Pal
2HQS, chains A, H – Xray TolB-Pal reference complex.

Give the following names:

- 1c5k.pdb (TolB, receptor protein)
- 1oap.pdb (Pal, ligand protein)
- 2hqs_A_H.pdb (reference complex)

In addition, you need an **".ini" file**, which contains the information about the chains to dock from each PDB file, in order to create a new pair of parsed PDB files suitable for *pyDock*. Thus, you have to create a text file called `T26.ini` using your favorite text editor and include the following information:

```
[receptor]
pdb = 1c5k.pdb
mol = A
newmol = A

[ligand]
pdb = 1oap.pdb
mol = A
newmol = B
```

The "*mol*" chain name is the original chain ID in the input pdb files whereas the "*newmol*" will be the new chain ID in the parsed pdb files suitable for *pyDock*. The "*newmol*" chain names must be different for the receptor and the ligand (so that you can distinguish the chains when docked!). The pdb in the ".ini" file must correspond to the exact names of the pdb files you have.

Once you have a complete `T26.ini` file, run the pyDock *setup* using this command in a single line:

```
pyDock3 T26 setup
```

This command will create the new PDB files for receptor and ligand `T26_rec.pdb` and `T26_lig.pdb` respectively. These files will be suitable for pyDock.

In case of PDB with incomplete side-chains, these will be automatically rebuilt with SCWRL 3.0 (`http://dunbrack.fccc.edu/`), which will be automatically launched within pyDock setup, in the pyDock class installation.

1.3 Perform the docking using ZDOCK

ZDOCK job can be submitted within pyDock but we will execute ZDOCK as we learned in the previous session:

```
/mnt/NFS_UPF/soft/zdock/IntelP3_Linux/zdock -R T26_rec.pdb -L T26_lig.pdb -o
T26.zdock
```

The docking will run from 10-15 minutes depending on the load of the computer. You can monitor the progress of the job using the command 'top -c'; look for the *zdock* process.

1.4. Transform ZDOCK output to pyDock rot file

Upon finishing the docking, you should have the following file in your directory: `T26.zdock`. Now, we need to transform the output data from ZDOCK (T26.zdock) to the rotation and translation matrix that complies with pyDock format. This file will include the rotation and translation information to transform the original coordinates of receptor and ligand in the docked conformation. This is done by using the following command:

```
pyDock3 T26 rotzdock
```

This calculation is quite fast, and it will create a `T26.rot` file containing the transformation matrices for all docking poses. Because of time limitations, in this practical we will only proceed with a subset of docking solutions. For that, you need to edit the `T26.rot` file to keep just 200 conformations.

IMPORTANT: Before editing the `T26.rot` file create a copy to save the entire set of docking solutions. For instance issuing this command:

```
cp T26.rot T26.rot.backup
```

The way you select the 200 docking solutions is up to you. You could for instance extract 200 randomly selected lines from the file using a scripts or the first 200 or the last 200 lines. A little of command line magic could do the trick. To select the top200 or bottom 200 simply do this:

```
head -200 T26.rot.backup > T26.rot
tail -200 T26.rot.backup > T26.rot
```

1.5. Scoring using the pyDock energy function

Next stage is to use pyDock energy function to score and rank all positions by running *dockser* module with the following command:

```
pyDock3 T26 dockser
```

*(you need to have in your working directory all the *pdb, *amber and *H files generated during pyDock setup)*

WARNING: Be sure you created a T26.rot file with just 200 conformations, otherwise this step would last several hours.

This step will take a few minutes. You can track the progress of the program on the terminal where you executed the command above.

When *dockser* finishes, take a look to output file called “T26.ene” that will look like the following example (with different values depending on the 200 docking solution selected):

Conf(1)	Ele(2)	Desolv (3)	VDW(4)	Total(5)	RANK(6)
8726	-28.979	-9.712	130.111	-38.691	1
4538	-28.001	-8.980	38.482	-36.981	2
6446	-29.716	-4.215	96.438	-33.931	3
1590	-32.394	0.109	28.699	-32.285	4

(1) Conf: conformation number of the docking pose (same as that in the *rot* file, last column)
(2) Ele: Electrostatic energy component
(3) Delsov: Desolvation energy component
(4) VDW: Van der Waals energy component (term weighted to 0.1 by default)
(5) Total: Total binding energy (Ele + Desolv + 0.1*VDW)
(6) RANK: conformation rank according to its computed binding energy

The ‘Total’ binding energy is the composite scoring function taking into account the contribution of electrostatic, desolvation and VdW components. Note the RANK, indicating the position among all the docking poses according to the pyDock energy function and not to the ZDOCK, even though the docking was done using ZDOCK.

1.6. Comparison with the real 3D complex structure

1.6.1 Generate the top10 docking poses

Now, you can select the most promising docking models taking into account pyDock energy ranking. You can compare the RMSD between the selected models and the real 3D complex structure (PDB 2HQS). Firstly, you have to generate the pdb file of your selected models as follows:

```
pyDock3 T26 makePDB 1 10
```

This will create pdb files for the docking poses ranked from 1 to 10, as indicated in the “*.ene” file, RANK column. Now check that your files appeared. They will be called as `T26_XXXX.pdb`, where *XXXX* is the conformation number as in the `T26.ene` file (*Conf* column; first column in the T26.ene file).

As in the previous cases you will need to load the coordinates of the docking poses and reference complex structure (`2hqs_A_H.pdb`) into Chimera and perform the same type of analyses as before.

ADDITIONAL EXPLORATION

Compare the results of pyDock scoring with the original ZDOCK scoring. Do you find any change in the number of near-native solutions within the top 10 scored models? Has the best near-native rank improved after pyDock scoring?

2. PATCHDOCK/FIREDOCK: SAVINASE-BASI COMPLEX

In this exercise we will work with the Savinase/Basi complex. Savinase is a secreted microbial serine protease and BASI is a Savinase inhibitor produced by plants. Hence, BASI has a role in plant defense binding to the protease (Savinase) and preventing its function. Indeed, BASI is produced by plants to protects the seed from pathogen-derived serine proteases including savinase.

In this exercise we will perform a rigid-body docking using PatchDock but then we are going to perform a limited refinement of rigid-body docking models with FireDock. As protein-protein interactions imply certain conformational changes, by doing a flexible refinement we hope to improve both the conformation of the models and the ranking of the docking poses. Important to stress that we will not be doing a fully flexible docking, just simple some localized optimization in the form of side-chain flexibility.

You can find the PDB files for this example in Aula Global (`SBI_17_P` folder / `SBI_17_P.zip`).

• savinase.pdb - The structure of the protease in PDB format (chain A).
• BASI.pdb - The structure of the inhibitor in PDB format (chain C).
• NativeComplex.pdb - The structure of the correct native complex (PDB 3bx1).

2.1. Rigid docking using PatchDock

2.1.1. Create parameter file for PatchDock

As before we are going to generate a parameter file (params.txt) using the following command in a single line:

```
buildParams.pl savinase.pdb BASI.pdb 4.0 EI
```

Note the parameter ‘EI’ stands for Enzyme-Inhibitor complex as such is this case.

2.1.2. Execute the docking

```
patch_dock.Linux params.txt out_file2
```

This step will take around 15 minutes to finish.

2.1.3. Examine several solutions

Generate top 10 docking models as explained in previous practical and analyze them as previously using Chimera.

```
transOutput.pl out_file2 1 10
```

Is there any good candidate among the selected ones comparing to the reference complex (NativeComplex.pdb)?

2.2. Refinement of the docking poses using FireDock

In this step, we are going to refine the docking poses generate by *PatchDock* applying limited flexibility. This step should in principle improve the docking poses and provide more realistic docking poses and a better ranking.

2.2.1. Protonate input structures for FireDock

FireDock require that input structures to be protonated, i.e., it needs to include the hydrogen atoms on each residue. There are different programs that can be used to protonate protein structures, since we don't have any available, we provided the protonated version of the input structures. You can find them in the `SBI_17_P` folder:

• savinase.pdb.CHB.pdb - Protonated structure of the protease in PDB format (chain A).
• BASI.pdb.CHB.pdb - Protonated structure of the inhibitor in PDB format (chain C).

2.2.2 Build transformation file from *PatchDock* output

FireDock uses its unique format for the transformation matrix (rotation/translation), which is different from PatchDock. You can change the format using this command in a single line:

```
/mnt/NFS_UPF/soft/FireDock/PatchDockOut2Trans.pl out_file2 > ex1.trans
```

WARNING: Be sure you extracted just the top 200 poses from `ex1.trans` ***file, otherwise the next step will be running for a long time!*** **For instance, like this:**

```
head -200 ex1.trans > tmp
mv tmp ex1.trans
```

2.2.3. Create parameter file for FireDock

Create parameter file (`paramsREF.txt`) needed by FireDock. In this case and given the time constraints, we will be doing a "coarse" refinement (RISCO, 50 cycles of RBO and radiiScale=0.8), with the option "1" to save the refined conformations to PDB files. Issue the following command in a single line:

```
/mnt/NFS_UPF/soft/FireDock/buildFireDockParams.pl savinase.pdb.CHB.pdb
BASI.pdb.CHB.pdb U U EI ex1.trans firedock.out 0 50 0.8 1 paramsREF.txt
```

2.2.4. Execute the refinement

Finally, execute the refinement with the next command in single line:

```
/mnt/NFS_UPF/soft/FireDock/runFireDock.pl paramsREF.txt
```

This step will take between 10-20 minutes depending on the CPU load. As before, you can track the progress of the program by inspecting the output in the terminal where you executed the command

2.2.5. Examine your results

Upon finishing two new files will be create:

`firedock.out.unref` (before refinement)
`firedock.out.ref` (after refinement)

These files contain information on the different scoring terms and refinement performed by FireDock. Of special interest is the 11th Column 'glob', which is the global score. There will be also 200 additional PDB files have been also generated upon the refinement of you docking poses selected from PatchDock (firedock.out_XXX.ref.pdb; XXX: given conformation) .

It will be necessary to sort the list of refined poses: `firedock.out.ref`. This can be easily achieved using the 'sort' Linux command as follow (in a single line)

```
sort -n -k11 firedock.out.ref > firedock.out.ref_sorted
```

The file `firedock.out.ref_sorted` will now contain the list of refined poses numerically (-n) sorted by the 11th column (-k 11). Remember that 11th column correspond to 'glob', which is the global score.

As usual, load the docking poses and the reference complex in Chimera and analyze them. Particularly to this exercise consider:

- Visualize the top 10 refined solutions (according to "glob" energy value) and compare them to the rigid-body docking solutions (before refinement) and to the reference complex (NativeComplex.pdb).

- Have the success rate of the top 10 docking solutions improved after refinement? i.e. have you increases the number of docking poses among the top 10 with smaller RMSD with respect to reference complex.

- Which rank and RMSD have the near-native solutions after refinement? (if there were not any near-native solutions in the rigid-body set, just apply this question to the best solution in terms of RMSD.

- Does full refinement provide better results than default (restricted) refinement?